

Embedded Systems Workshop

Audio Guided Microbot

3-12-2025

Team: Sensodyne



Team no: 17



Ayush, Pranav,
Yashav, Raza



Under Prof. Abhishek
Srivastava



TA: Inesh Dheer

Problem Statement

The aim of this project is to design an **intelligent autonomous microbot** system equipped with a **MEMS microphone** and will be capable of detecting and identifying commands (edge computing) in **real time** and take action against it.

The system will include **secure wireless** communication for remote operation, and a **comprehensive GUI-based** dashboard to **monitor and control** the microbot.

Problem Statement

The aim of this project is to design an **intelligent autonomous microbot** system equipped with a **MEMS microphone** and will be capable of detecting and identifying commands (edge computing) in **real time** and take action against it.

The system will include **secure wireless** communication for remote operation, and a **comprehensive GUI-based** dashboard to **monitor and control** the microbot.

- **Networking:**

Heavy amounts of classification of different communication methods to choose the most reliable and secure option for communication link.

- **Server & Dashboard:**

Single GUI to monitor status and logs, switch modes, and allow instant operator takeover with clear state.

- **Audio Classification:**

On-device, real-time recognition of a small command set; robust to noise/speaker variation with low false triggers.

- **Movement:**

Compact mobility with precise maneuvers; automatically halt forward motion within a defined obstacle range.

Problem Statement

(broken down)

- **Reliable Voice-Command Guidance**
- **Secure wireless communication (BLE / LoRa)**
- **Should have Manual Override**
- **Obstacle Detection and Avoidance**
- **Low footprint
(small, low-power, low-cost)**
- **GUI to keep track of the bot
(battery, logs, control)**

Motivation

Hands-free Control

Voice makes operation simple for quick tasks and improves accessibility.
Eg. Wheelchair automation, household transport

Works offline & private

Edge KWS on ESP32-S3 avoids cloud latency and protects audio data.

Library monitoring

Autonomous navigation around a fixed path, and generates a heatmap for noise across the library

Safety & supervision

Obstacle stop + PC telemetry/logs for accountability in places such as Factories.

Spy Bot

Recognizes text around a bugged spot, and sends it wirelessly and reliably over a large time period of time

POC for AI computational model on edge

Reproducible build for labs, hackathons, and student research.

☐

☐

☐

☐

☐

☐

The solution

The Concept: An autonomous microbot that uses **Edge Computing**. It thinks and acts locally.

Core Capabilities:

- Real-time Keyword Spotting (KWS).
- Obstacle Avoidance (Safety).
- Secure Wireless Telemetry (BLE/LoRa)

Use Cases: Wheelchair automation, industrial monitoring in privacy-sensitive areas, or "Spy Bots"



Edge AI KWS using CNN

Size

Raw Model Size: ~500KB

Quantized Size: ~81KB

Can run on ESP32 Wroom

Key Parameters:

- 77,473 trainable parameters (99% in Dense layer)
- INT8 quantization: 500KB → 81KB (4× compression).

Compression Ratio: 1.07x (TFLite)

CNN Architecture and Optimization

- Average response time being 102ms (only 2ms above minimum value, very good values)
- Minimalist CNN: Only 2 conv layers (4 filters) + 2 dense layers
- INT8 Quantization: Converts 32-bit floats → 8-bit integers (4× compression)
- Has about 90-95% accuracy of detection when tested in hostel rooms. (drops significantly in crowded rooms). (optimizations given next)
- Used about 75,000 samples to train in total.

Spectrogram Processing

Turning Voice into "Image":

- Audio split into 20 ms windows, FFT extracts frequencies (like equalizer bars)
- Creates 99×43 grid: time (horizontal) × frequency (vertical)
- CNN treats this as image; For Example "forward" is accurately differentiated from "left" because of its different visual nature.

Edge AI: KWS using CNN

Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
conv_layer1 (Conv2D)	(None, 99, 43, 4)	40
max_pooling1 (MaxPooling2D)	(None, 49, 21, 4)	0
conv_layer2 (Conv2D)	(None, 49, 21, 4)	148
max_pooling3 (MaxPooling2D)	(None, 24, 10, 4)	0
flatten_1 (Flatten)	(None, 960)	0
dropout_2 (Dropout)	(None, 960)	0
hidden_layer1 (Dense)	(None, 80)	76880
dropout_3 (Dropout)	(None, 80)	0
output (Dense)	(None, 5)	405
=====		
Total params: 77,473		
Trainable params: 77,473		
Non-trainable params: 0		

CNN model: 9 layers, ~77.5k parameters

5 layers dedicated to robust audio pre-processing

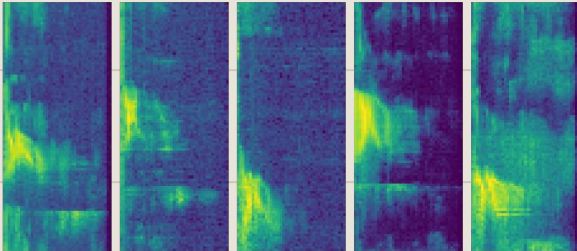
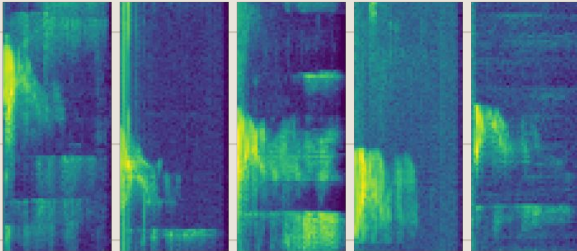
Commands: *forward, back, left, right*

On-device smoothing: *confidence threshold + debounce to reduce false triggers*

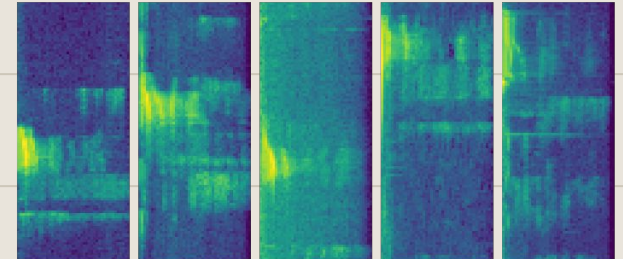
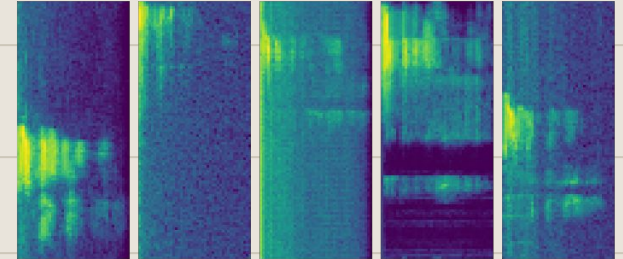
Runtime: *Fully on-board (no internet) → low latency, consistent behavior*

Spectrogram Parameters

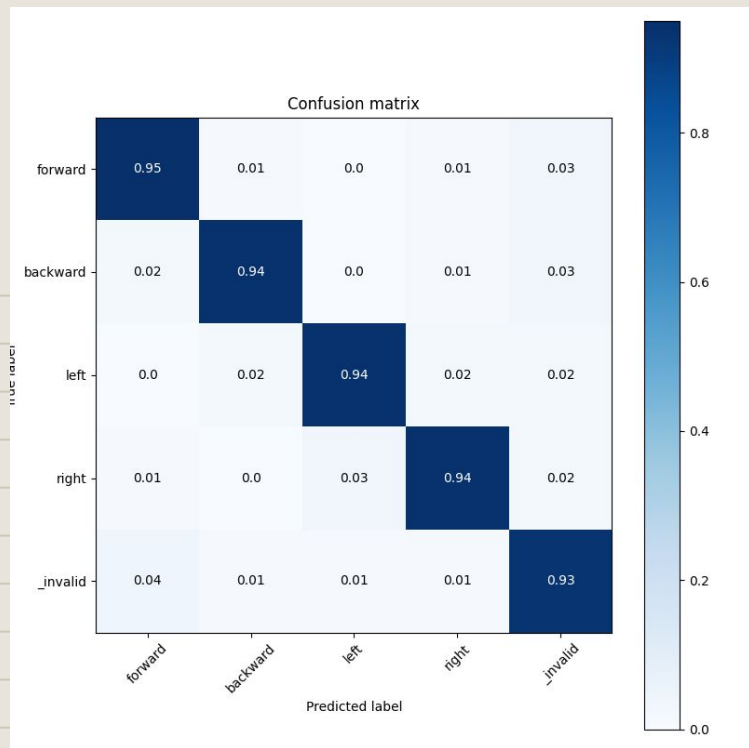
This is the Spectrogram for the keyword “right”. Where it is clearly observed that it has all it has a **shorter** frequency range compared to left



This is the Spectrogram for the keyword “left”. Where it is clearly observed that it has all it has a **wider** frequency range compared to right



Confusion Matrix (Latest Dataset)



We have tried to optimize this model to **avoid false positives**, especially in crowded environments, and this matrix is proof of that information.

The key rule is having a lot of compression layers and a **single dense layer** for least confusion, this also indicates **overfitting** of data, which is not a problem in this case since our keyword count is very limited, and used a massive open source dataset. About **75,000** samples are used. (Uses TFLite)

Link for more details on model:

<https://docs.google.com/document/d/1q8NcpLEvE-21mCjpiRTARdAPb8YxRwf7LxyK7FXPdC0/edit?usp=sharing>

Implementation Challenges

- Mic not being consistent with time
 - Having a lot of false positives towards the end of timeline due to the possible reasons
 - Noise from PCB connections
 - Mic having unregulated voltage
 - Dust collection
- Cannot make the Model larger otherwise it would start eating up FreeRTOS memory
- Confusion, and Accuracy Matrix are theoretical values only
- Previously recorded Real Accuracy values seem to have decreased significantly
- All the above implies addition of extra **Noise** in the mic input making our bot move like a ghost (false positives)

Noise Filtering For Mic Inputs

Added A gating mechanism

- To reduce ambient sound/noise readings from surroundings or mic leading to false positives, we gate the inputs to the model based on input thresholds which will give only **possible** words to the model leading to significantly lesser false positives
- Set by real life testing and approximations

```
constexpr float kSilenceScore = 1e-6f;
constexpr float kProbabilityThreshold = 0.55f;
constexpr float kProbabilityMargin = 0.20f;
constexpr float kProbabilityThresholdStrong = 0.45f;
constexpr float kProbabilityMarginStrong = 0.15f;
constexpr unsigned long kDetectionCooldownMs = 1000;
constexpr unsigned long kDebugLogIntervalMs = 200;

constexpr float kSpeechGateSnrThreshold = 1.05f;
constexpr float kSpeechGateStrongSnr = 1.8f;
constexpr float kSpeechGateMinRms = 45.0f;
constexpr float kSpeechGateStrongRms = 90.0f;
constexpr float kSpeechGateMaxZeroCross = 0.28f;
constexpr float kSpeechGateZcrMargin = 0.08f;
constexpr float kSpeechGateMinAbsDelta = 5.0f;
constexpr int kSpeechGateMinHits = 2;
constexpr int kSpeechGateHangoverHits = 4;
constexpr unsigned long kSpeechGateReleaseMs = 600;
```

Best Score Filtering

- Once the gate passes an audio sample, Then for each of these commands, we set a **custom threshold** for the probability vector of the output to determine its action.

```
constexpr float kCommandProbabilityThresholds[NUMBER_COMMANDS] = {
    0.55f, // forward
    0.55f, // backward
    0.55f, // left
    0.48f, // right (slightly lower)
    0.60f // unknown/silence slot stays strict
};

constexpr float kCommandProbabilityMargins[NUMBER_COMMANDS] = {
    0.20f,
    0.20f,
    0.20f,
    0.15f,
    0.25f
};
```

Data Sample Filtering

Noise-Threshold Trimming

- Locates where real speech starts/ends by ignoring low-level background noise.
- Normalize the waveform, mark samples above a noise threshold, and take the first and last such indices as speech bounds.

Voiced-Segment Length

- Measures how much of the clip is actual speech instead of silence/noise.
- Subtract the speech start index from the end index to get the number of speech samples.

length + speech filter

- Accepts only clips that are the right size and contain enough clear speech.
- Confirmed if the length is correct, and has speech words in the file (non-empty)

minimum speech duration

- Rejects clips with too little speech, which would confuse training.
- Finds the speech starting and ending bounds to measure the speech length.

Extra Calibration Checks

Audio Path I2S mic

Additional calibration checks were performed on the audio path and I2S microphone, focusing on input gain, noise floor capture, and verification of the feature-scaler freeze from training.

KWS Thresholds

Calibration included tuning the confidence threshold, debounce window, and command cooldown parameters for reliable keyword detection.

UltraSonic Sensor

Calibration involved verifying the distance threshold for stop detection using ruler-based measurements.

Locomotion

Calibration focused on establishing the PWM-to-speed mapping and adjusting left/right trim to ensure straight-line motion.

Audio Classifica tion

Scenario	Speaker db	Microbot db	Ambient db	Distance
Little ambient noise	88	66	60	1.25m
Little ambient noise	90	68	61	2.6m
Little ambient noise	91	67	60	5m
Noisy (multiple conversations)	92	72	68	2m
Noisy (multiple conversations)	93	76	70	5m
Very Noisy	95	82	77	1.5m
Echo bound	92	73	65	2m

To evaluate the audio recognition capacities of our bot we did classification in different scenarios, we measured the decibels near the speaker, and near the microphone on the microbot, we also measured the ambient decibels in the setup and the distance between the speaker and the bot. For each set up we measured the values multiple times and took the average and rounded off to the nearest integer.

Sensors & Actuators



INMP441 I²S mic

(For Voice input)
Using **mono audio**
(Left only) stream to get
clear audio, and easy to
use.
Confirmed the working of
The mic using a live
datastream player from
mic to laptop speaker.



Ultrasound HC-SR04

Used for **obstacle detection** in front of the bot, and decides its motion respectively. Currently set to **6cm**.

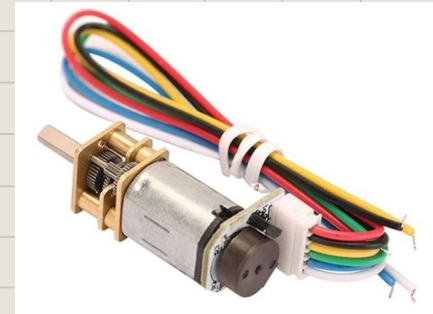
Verified the working of components beforehand using simple programs and an ESP32 along with a ruler.



N20 Motors

Designed a differential drive system using n20 motors and a ball wheel to steer the microbot.

It is a 6v 15rpm motor with encoder (tested and calibrated physically) along with high torque factor to support heavy batteries.



Components

ESP32-S3 & ESP32

Chose the much more powerful ESP32-S3 to play around and tune the model to make it optimised for vector multiplication acceleration.

And since we are compiling for FREERTOS directly so simulating it would be challenging.

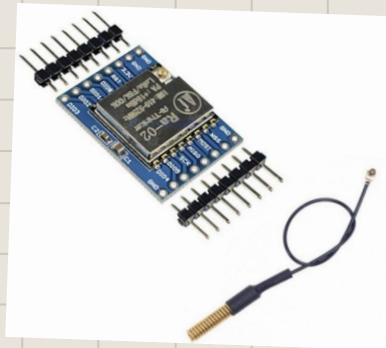
Eventually the model can run on ESP32 Wroom as well.



LoRa module + inbuilt BLE

Using the inbuilt BLE antenna and software within the variety of ESP's for a more reliable connection for short range.

LoRa is having much higher range but it also comes at a cost of connectivity, and bandwidth (slightly higher bandwidth reduces our range)



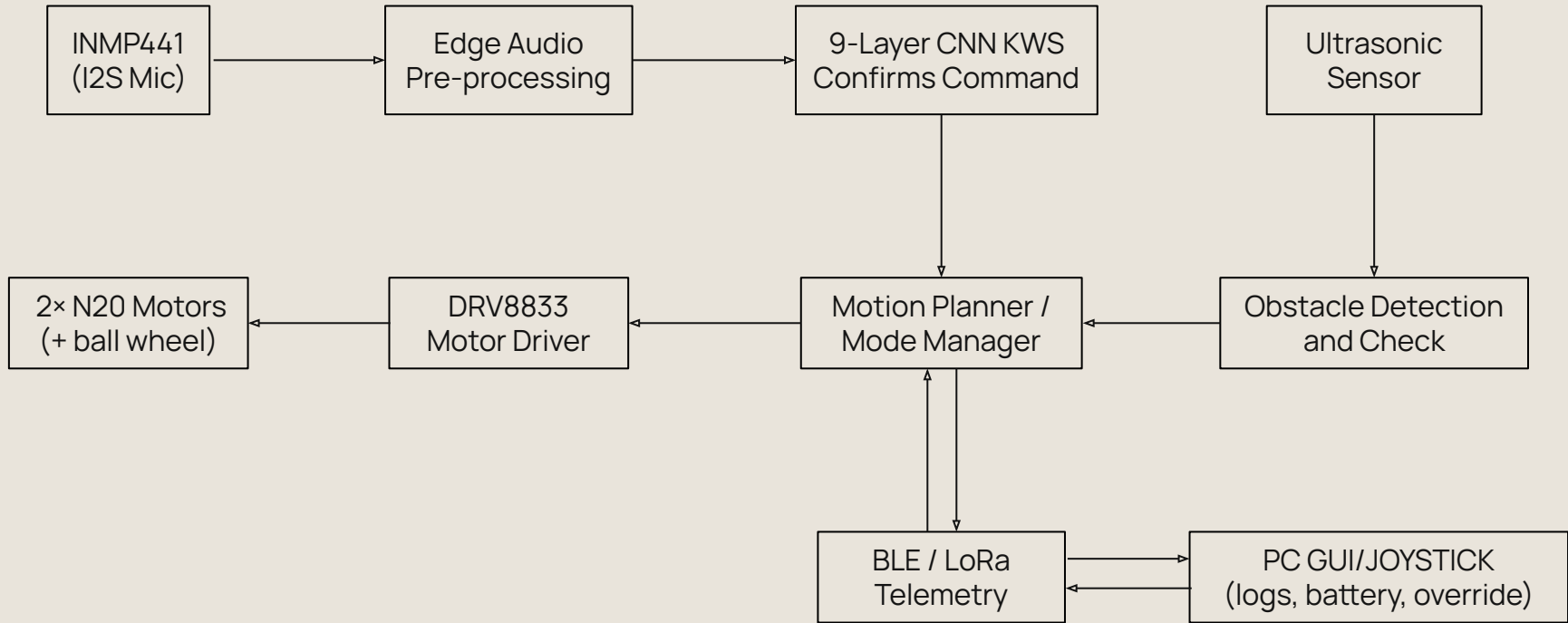
Battery + voltage sensor + current sensor + buck converter

Using the above components, these combine to give a voltage regulated power supply to the esp and motors to make the bot truly autonomous.

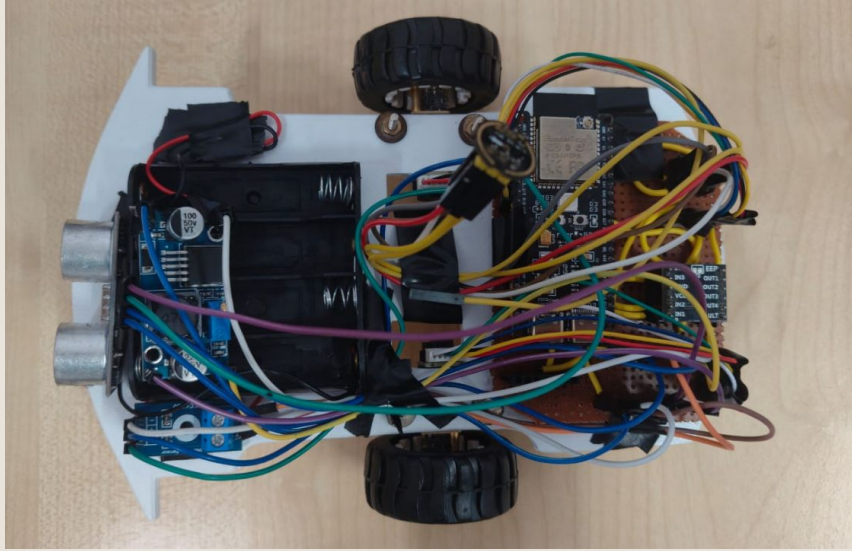
(not implemented for the demo due to logistical issues)



Block Diagram

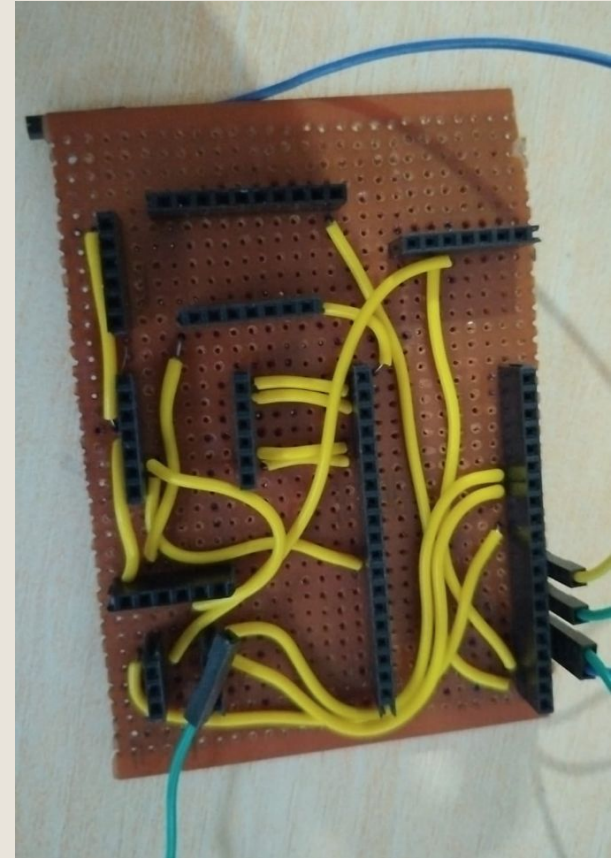
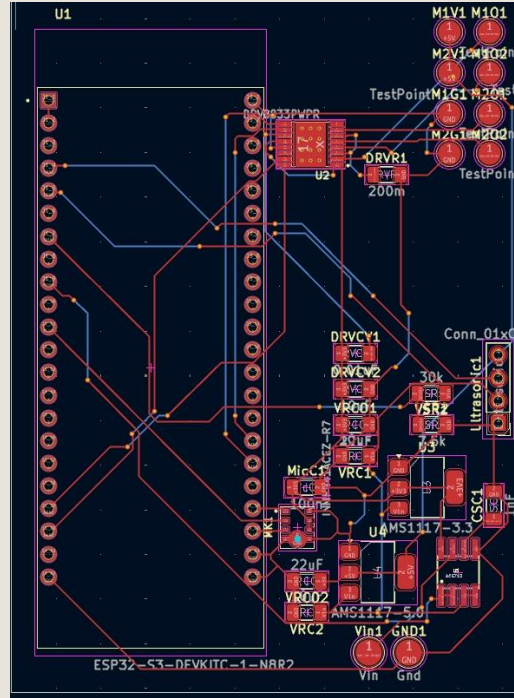
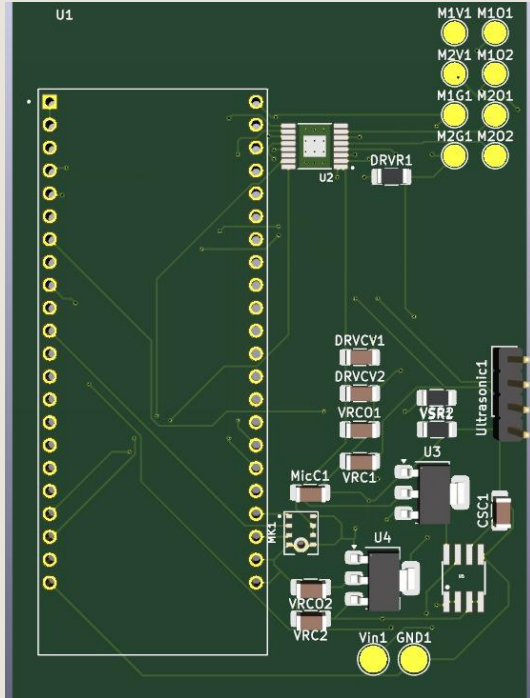


Hardware (The Microbot)



PCB

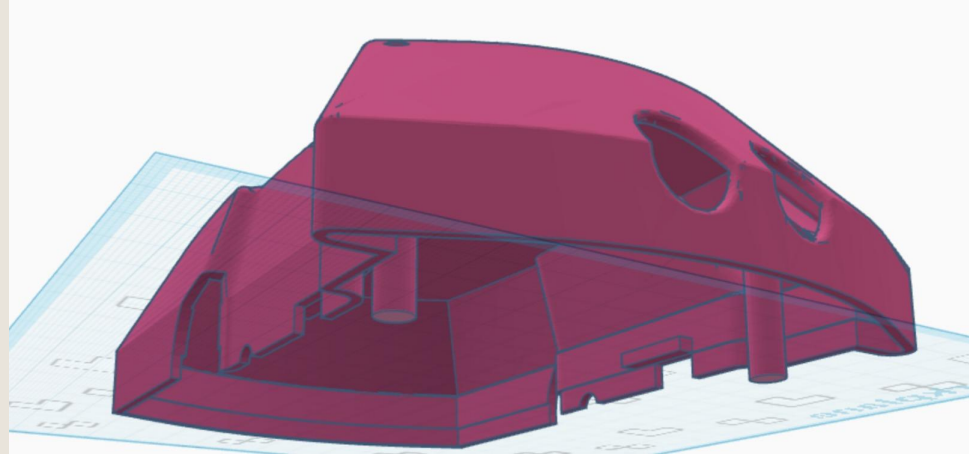
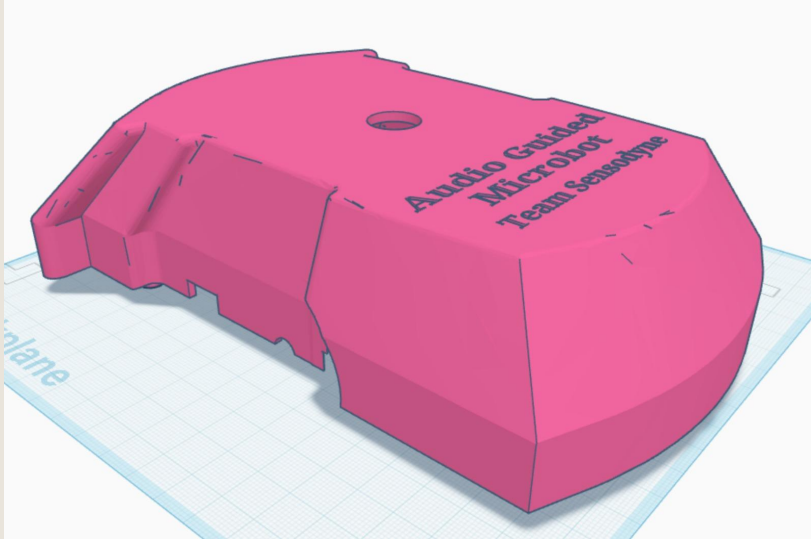
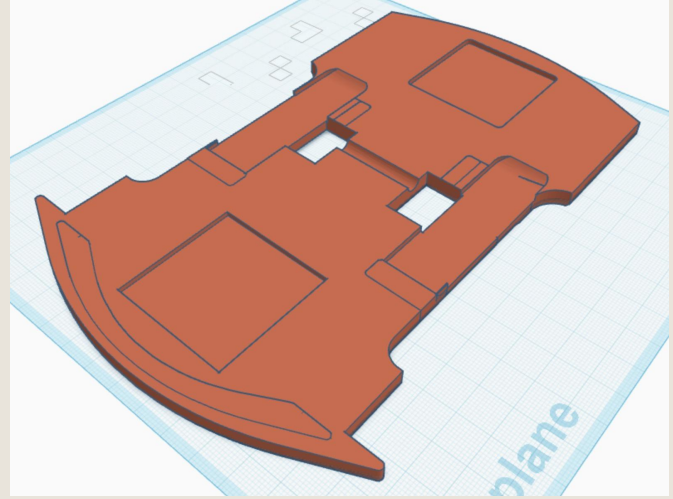
Direct implementation using the circuit diagram, made using KiCad.



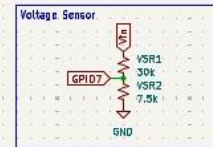
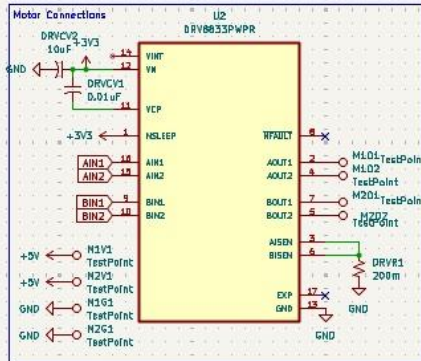
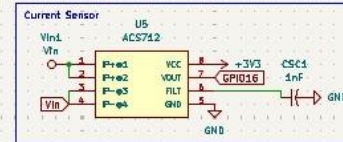
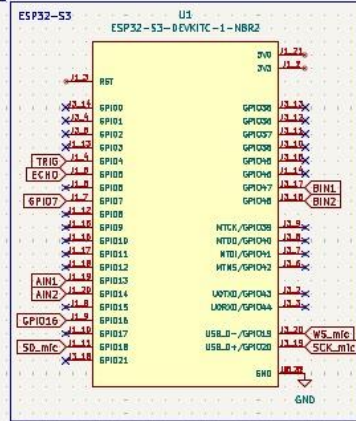
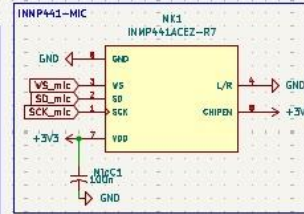
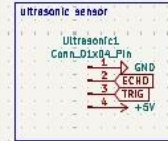
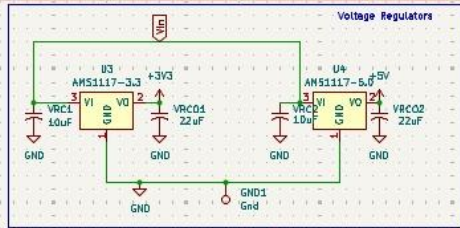
3D Model

Took inspiration from online sources and went through many websites to find the most suitable and purposeful design for the audio guided autobot.

The final design was made using TinkerCAD using accurate measurements and making it as compact and accessible as possible.



Circuit Schematic



Battery Life

Motor Runtime	Avg. Current (A)	Runtime (h)	hh:mm
0%	0.18	~3.50	~3h 30m
25%	0.198	~3.00	~3h 00m
50%	0.215	~2.75	~2h 45m
75%	0.232	~2.50	~2h 30m
100%	0.25	~2.25	~2h 15m

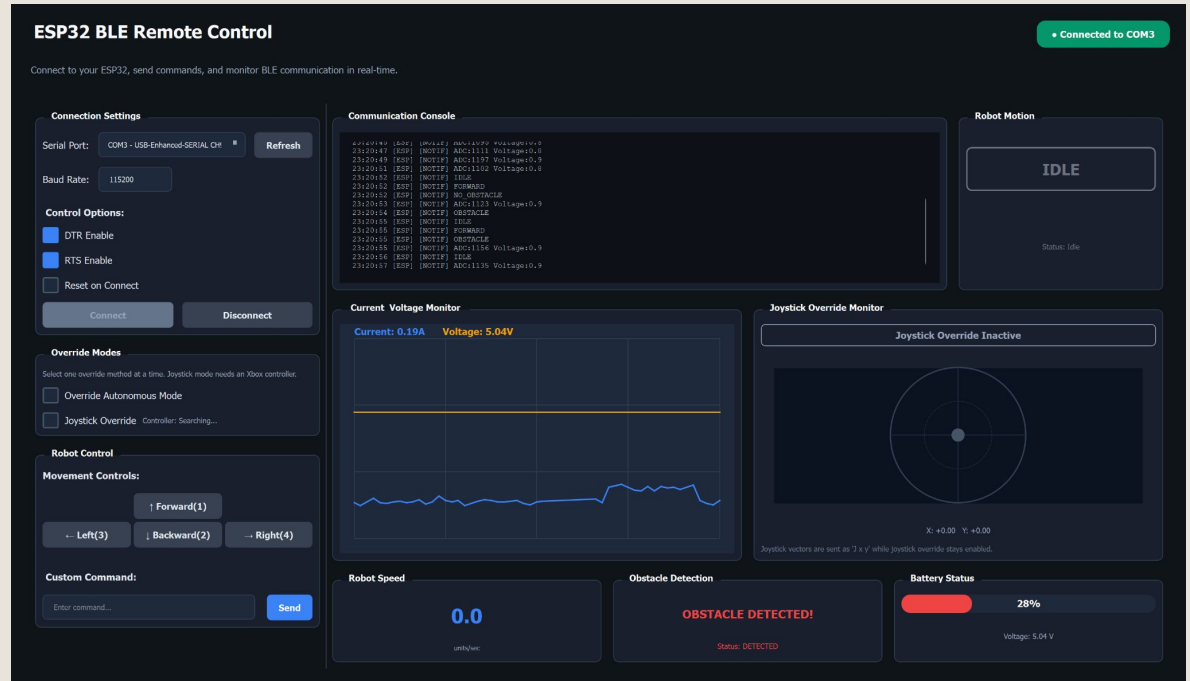
To evaluate the endurance of our microbot, we measured its current consumption at different levels of movement activity. As expected, increased motion leads to a proportional rise in average current draw, which directly shortens the available runtime.

These tests were conducted using 4 AA batteries (each of 1.3V with 600mah capacity) with total 5.2V (in series) under consistent operating conditions, with the microbot performing repeated movement cycles to ensure reliable measurements.

For better battery life,

Remote GUI

- Connects to the ESP32 microbot over serial and shows real-time logs.
- **Manual Override:** toggles voice control off and hands control to the GUI.
- **Drive controls:** buttons & arrow keys for Forward / Backward / Left / Right, or Use a **Controller**
- Shows motion state (color-coded: FORWARD / BACKWARD / LEFT / RIGHT / IDLE).
- **Live telemetry:** battery voltage and current draw with a small live graph.
- Displays approximate speed (no-slip estimate).



- **Obstacle status:** clear vs OBSTACLE DETECTED (auto switches to IDLE on detect).
- Send custom commands directly from the input box.
- Also shows the joystick input after validation, and transmits it successfully

Joystick Implementation

An Extra Feature of to bot: Real-Time Bot Control Using Joystick Interface

- Added **real-time robot motion control** using a Bluetooth-connected joystick for precise movement.
- Implemented a **Python GUI** to read joystick values and visualize motion.
- GUI sends **continuous 2-axis vector commands (Jx,y)** over BLE.
- ESP32 converts joystick vector into **differential motor speeds**.
- Enabling us to control the bot in 8 directions, and analog speeds

Read **actual joystick hardware** (controller), analysing and plotting it in gui and then sending the vectorized signals (Jx,y) to the microbot for actual motion via the Bluetooth connection, everything done by using python libraries (like Pygame).



- Measuring X and Y coordinate between -1 and 1 for precise movement

Direction of motion of motors is calculated according to coordinates.

Converting motor speed to PWM:

PWM Duty= |Speed|×255

(Positive value → forward pin

Negative value → reverse pin

Zero → motor stops)



- Smooth acceleration and turning.
- Full 360° joystick control.
- Differential turning (tank-like steering).
- Live motion with **no noticeable delay**.
- Automatic stop when no joystick data for 400ms (safety timeout).



Joystick → (x,y) → simple formulas → left/right speeds → PWM → motors

Network Communication Classification

LoRa v/s BLE



To help decide the network protocol in the early stages of the project we performed Characterisation of LoRa and BLE communications. We tested the **signal strength** and **communication reliability** in different setups, which took into consider factors that could cause disturbance in communication like **distance, walls, metal doors, echo, and interference from other devices.**

BLE Classification Inferences (with top cover)

DOS- Direct Line of Sight. Using esp32 and esp32S3.

Range(m)	Inference
0m (DOS)	✓ Reliable Communication
30m (DOS)	✓ Reliable Communication
30m (echoey hallway) (DOS)	✓ Reliable Communication
50m (echoey hallway) (DOS)	✗ Limiting Communication
70m (DOS)	✓ Reliable Communication
100m (DOS)	✗ Limiting Communication
20m (two walls in between)	✗ Limiting Communication

BLE Classification Inferences (with top cover)

DOS- Direct Line of Sight. Using esp32 and esp32S3.

Fast movement: running, Slow movement: approx. speed of bot

Range(m)	Inference
Start at 0m and fast movement upto 30m	✓ Reliable Communication
Fast movement upto 50m	✗ Limiting Communication
Slow movement upto 60m with no DOS.	✓ Reliable Communication
Slow movement with DOS upto 85m	✓ Reliable Communication
15m (2 metal doors)	✗ Communication disrupts as 2 nd door introduced
Introducing movement and objects at 100m	✗ Unstable Communication

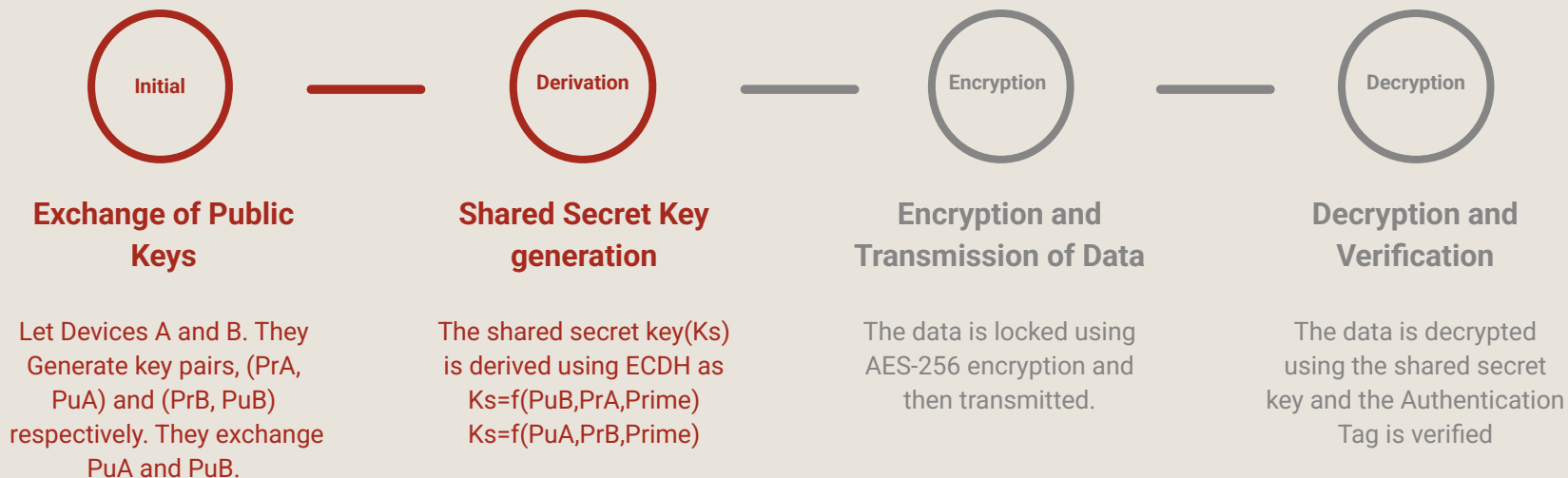
LoRa Classification Inferences

DOS- Direct Line of Sight

Range(m)	Inference
0m (DOS)	✓ Reliable Communication
15m (3rd floor to ground floor)	✗ Unreliable Communication
20m (2 metal doors)	✓ Reliable Communication
50m (few walls in between)	✓ Reliable Communication
110m (echoey Hallway)	✗ Unstable Communication
100m (DOS)	✓ Reliable Communication
175m (DOS)	✓ Reliable Communication
254m (DOS)	✗ Communication Terminates

Encrypted LoRa Communication

To ensure secure communication we performed encryption over LoRa communication. We used Elliptic Curve Diffie-Hellman and AES-256 encryption. Each device creates a **unique pair of keys**, one private and one public. The two devices exchange their public keys and derive a shared secret key using **ECDH (Elliptic Curve Diffie-Hellman)**. The shared key is used for **AES-256 encryption** to lock data before transmission. The receiver unlocks and verifies the authentication tag.



Challenges Faced

Change in problem statement

- Our initial approach required detecting the direction of sound using multiple microphones and applying beamforming, vectorization, and triangulation.
- Beamforming is complex because it depends on precise **time-of-arrival** differences in sound between microphones. Even tiny errors in spacing, synchronization, or noise break the calculations.
- All mics need to be **calibrated** correctly and **synchronized** which is very challenging.
- Our hardware setup (ESP32 + low-quality mics + no synchronized ADC sampling) was not capable of the precision needed.
- Debugging was extremely time-consuming because direction detection needs:
 - calibrated microphone spacing
 - stable phase alignment
 - high-quality simultaneous audio sampling
 - clean signals with minimal noise
- After spending a major portion of time, we still couldn't get reliable direction estimates, and the problem statement itself was later changed, making this entire effort a heavy time investment with no final output.

Challenges Faced

ML Model Creation & Edge Impulse Failures (Audio-Guided Microbot)

- Due to ESP32 memory limits, our model had to be extremely small (few kB), which made training very restrictive.
- We tried multiple solutions – **custom datasets, pre-trained models, and many Edge Impulse pipelines** – but most models either did not fit or had insufficient accuracy.
- The Edge Impulse model worked perfectly on their website and in the browser test, but the moment we deployed it on the ESP32, it stopped working.
- To debug this, we streamed **raw mic audio to a laptop via serial** and tested with:
 - Python audio playback (proved the mic input was OK)
 - Whisper-tiny in real-time (also worked well)
- This confirmed: the mic and data were fine, the ESP was the issue.
- Despite all this, we never found the root cause of the failure on the device – the model simply refused to run properly once flashed.
- This consumed a massive amount of time across many days.
- (We eventually solved it using a manual .tflite model + TFLite-Micro, so the solution is not listed as a challenge – the difficulty was the long process before reaching it.)

Challenges Faced

Microphone Noise & Threshold Tuning

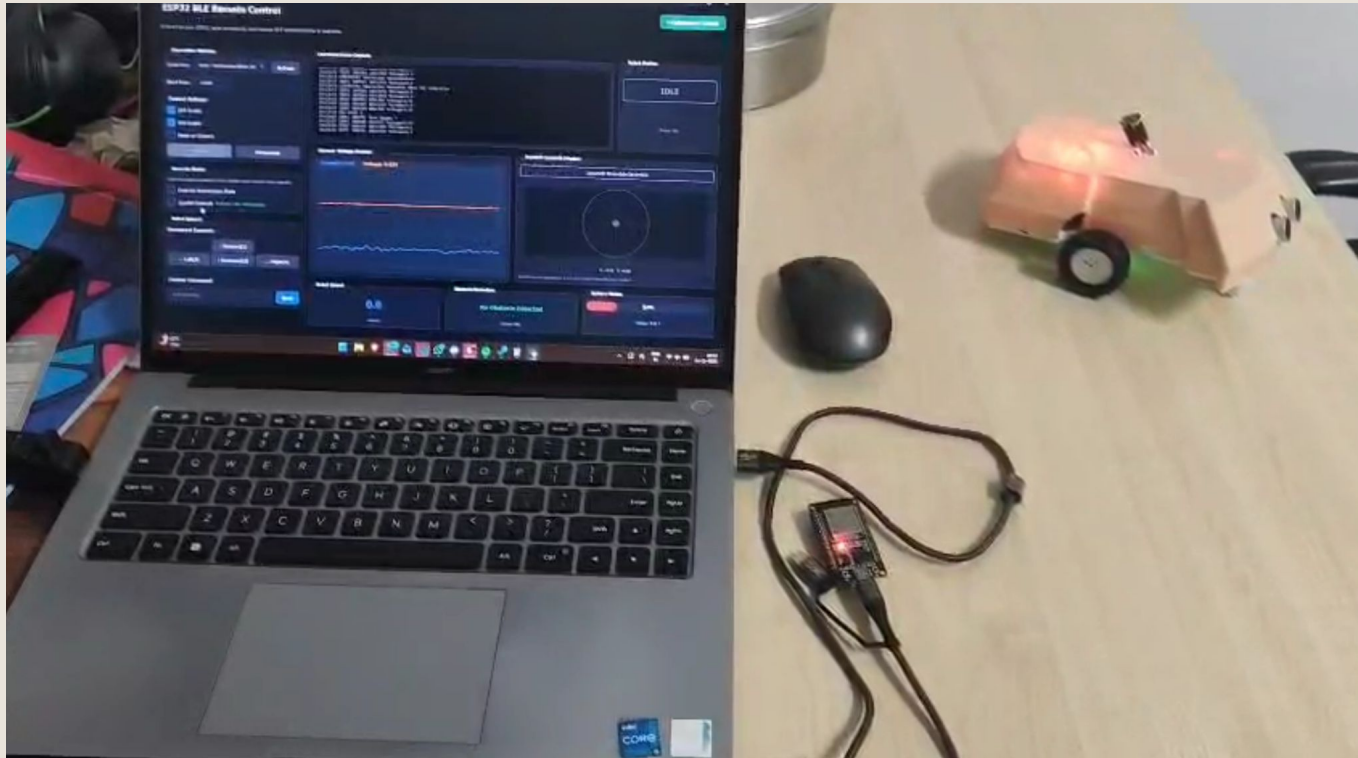
- The microphone produced unpredictable noise (possibly PCB/soldering issues/dust accumulation).
- Noise caused **false positives**, and raising thresholds caused false negatives — tuning was **extremely sensitive**.
- This instability made the model output unreliable.

Early Hardware Instability (Breadboard Phase)

- Wires frequently **disconnected** during tests.
- One breadboard power line was internally faulty, causing random non-working circuits.
- Loose connections slowed debugging significantly.

Joystick Control Issue (GPIO Conflict with NeoPixel)

- On the ESP32-S3, both the motors and the NeoPixel LED were using GPIO 48.
- For simple commands (forward/back/left/right), both worked fine.
- But during joystick mode (full 360° control + dynamic RGB color), the NeoPixel timing interfered with motor control, causing the **motors to stop working**.
- Removing the RGB LED (not needed once we added an enclosure) fixed the issue — motors then **responded perfectly**.



Demo Video Link

<https://drive.google.com/file/d/14PTspYeGjPHkEY1JN1xzq3BRwo0eiGK6/view?usp=sharing>

THANK YOU!

(Don't forget to brush)